

4A. Práctica de tratamiento de señales en sistemas embebidos con Python

Alumno: Claudio Iván Esparza Castañeda

Profesor: Luis Alberto Balam Guzmán

Objetivo

Lo que hizo Arduino por la cultura maker fue fundamental para democratizar la tecnología. Por otro lado, otro impulso se dio a través del uso de Python por la misma comunidad (junto a su creador, Guido Van Rossum). En algún punto estas dos fuerzas convergieron para llegar a micropython, que es un proyecto que empuja hacia esta democratización para el tipo de usuario que en su día a día no tiene tiempo de aprender todo acerca del hardware, pero busca realizar proyectos de fin de semana, o bien el profesional o el estudiante que requiere un prototipado rápido para el proyecto de fin de semana. Después del Arduino, han aparecido diferentes soluciones con distintos controladores. Así es como llegamos a la Raspberry Pi Pico, que se puede programar a través de bare-metal, con RTOS (FreeRTOS, Zephyr, etc.), o micropython.

Con la imposibilidad que representa tener que procesar grandes cantidades de información en un medio con pocos recursos, refiriéndonos en este caso a la Raspberry Pi Pico, surge la necesidad de contar con sistemas híbridos: control en el sistema embebido, y procesamiento en la computadora.

Descripción de la solución

Se sigue la estructura propuesta para el proyecto. Hay un único cambio realizado en el archivo `generador_serial.py` fue renombrarlo a `main.py` e incluirlo en la carpeta `pico/`, Este cambio se realizó con el objetivo de poder usar el puerto serial sin que se bloquee el puerto serial por el REPL. Por otro lado, se dividieron responsabilidades a través de las distintas partes requeridas. Por un lado, el archivo `captura.py` se autodescribe. Se encarga de establecer la conexión del puerto y recibir la información procedente de la tarjeta. El archivo `procesamiento.py` también se autodescribe, contiene las operaciones científicas de los módulos de Python, de acuerdo a lo requerido. El archivo `visualizacion.py` contiene las funciones encargadas de la creación de las gráficas.

Se sigue usando el archivo `funciones.py` de tareas anteriores, con el objetivo de hacer algunas validaciones. El archivo `main.py` se encarga de ejecutar todo lo anterior en el orden dado.

Metodología

El tratamiento de las señales se realiza con SciPy, al emplear las funciones definidas en este módulo. Primeramente se encuentra un filtro pasa-bajas tipo Butterworth de 4 orden. Se comprobó que no hay mucha diferencia entre el filtro de segundo orden, tercero ni quinto. entre mayor sea el número de muestras conviene un filtro de mayor orden, pero para 1000 no hay una diferencia apreciable.

Para la detección de picos primero se calcula la desviación estándar de la señal filtrada, y se emplea la función `find_peaks` incluida en SciPy

Para la detección de anomalías se calcula el residuo como la diferencia entre la señal original y la señal filtrada. Después se encuentra la desviación estándar y se obtiene los que están fuera del residuo en el orden de la desviación estándar.

Por último, el análisis FFT utiliza las funciones `rfft` y `rfftfreq` para obtener las magnitudes y

frecuencias de manera casi directa.

La visualización, por su parte, sólo es emplear las salidas del procesamiento para establecer los ejes de las gráficas.

Capturas de ejecución

```
~/Documentos/Escuela/INFOTEC/Programación y procesamiento de datos/Programacion_y_Procesamiento_de_Datos_Infotec/4A main* 17m
> python main.py
=== Sistema híbrido - Laptop/Raspberry Pi Pico ===

Cantidad de muestras a capturar (mínimo 1000): 1000

Comunicación serial
Conectando a /dev/ttyACM0 @ 115200 baudios...
100/1000 muestras capturadas...
200/1000 muestras capturadas...
300/1000 muestras capturadas...
400/1000 muestras capturadas...
500/1000 muestras capturadas...
600/1000 muestras capturadas...
700/1000 muestras capturadas...
800/1000 muestras capturadas...
900/1000 muestras capturadas...
1000/1000 muestras capturadas...
Captura completa: 1000 muestras válidas, 0 descartadas por no ser numéricas.

Datos guardados en 'datos/mediciones.csv'.

El csv se guardó correctamente
Tratamiento de señales con SciPy
Picos detectados: 14
Anomalías detectadas: 14

Creación de gráficas
Gráficas guardadas en la carpeta 'resultados/'.

Proceso finalizado.
```

Figure 1: Ejecución del programa. Se pide de entrada el número de muestras a capturar..

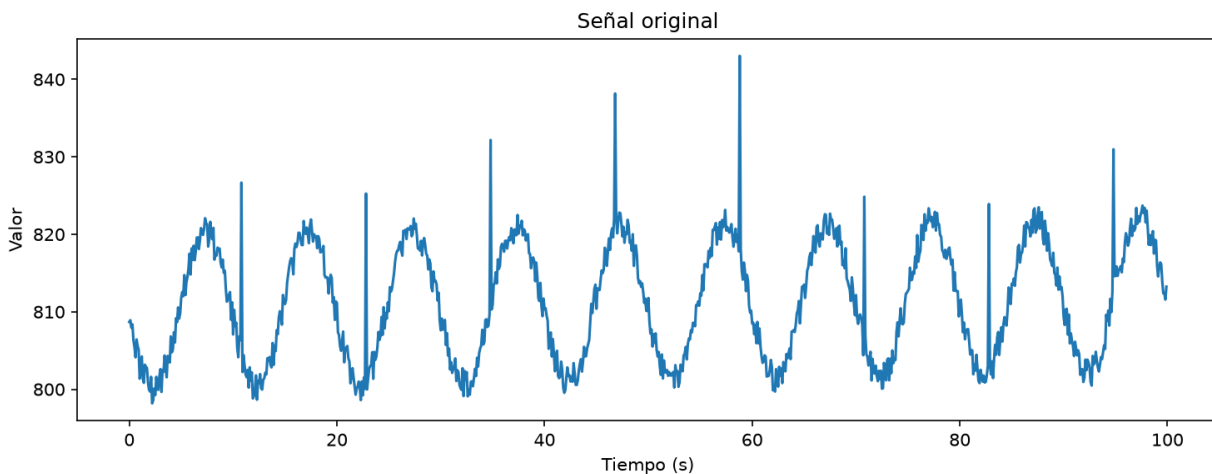


Figure 2: Señal original.

Conclusiones

MicroPython permite crear prototipos rápidos para interactura con hardware y tomar mediciones del ambiente. Con esto permite su uso en el mundo maker, artístico e ingenieril, por mencionar algunos. Trabajar de manera híbrida permite que la placa se encargue del control y adquisición de datos, mientras que la computadora, más potente, se encargue del procesamiento.

La carpeta en el repositorio de este proyecto es: Programación y procesamiento de Datos.

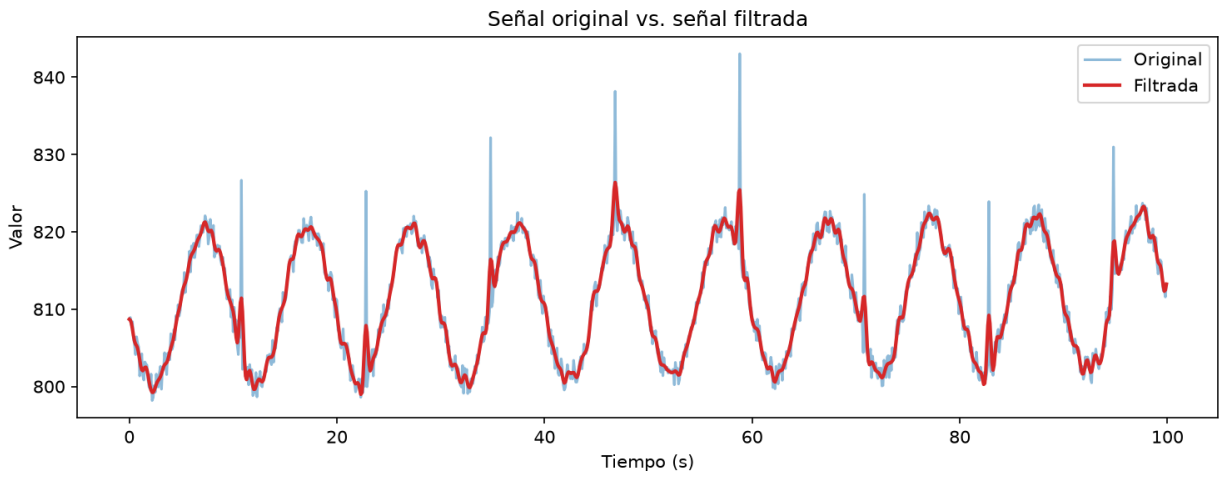


Figure 3: Señal original vs. señal filtrada

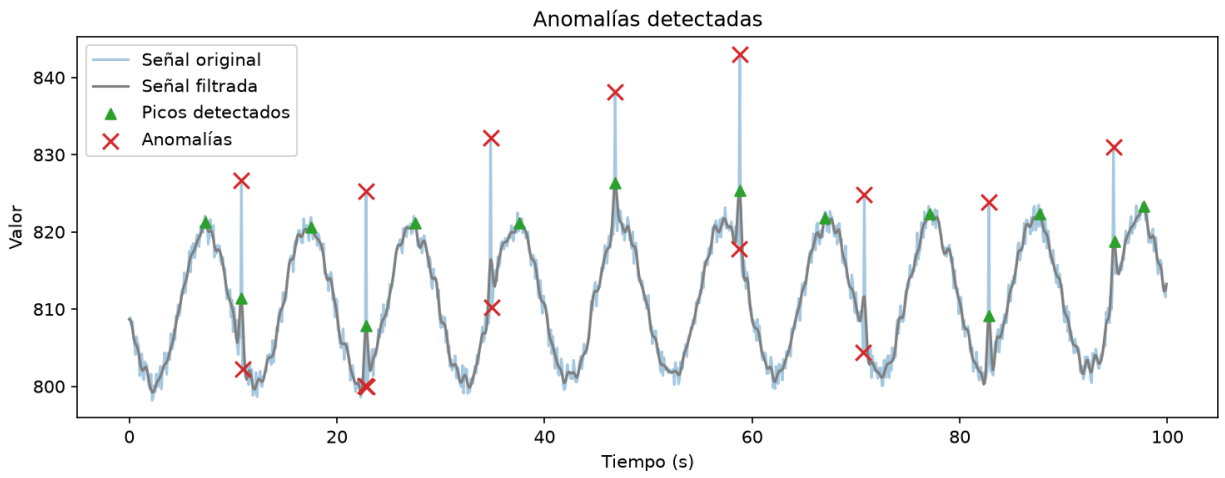


Figure 4: Anomalías detectadas.

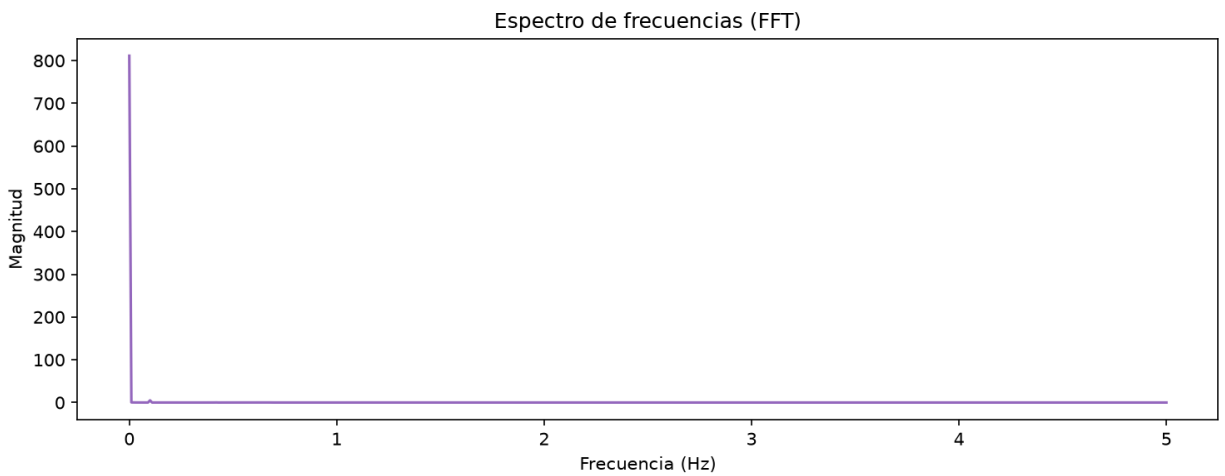


Figure 5: Transformada rápida de Fourier (FFT).